

Milestone 1 — *HTML, CSS & GitHub Fundamentals*

◆ Getting Started — VS Code & HTML

What is HTML?

HTML (HyperText Markup Language) is the language used to structure content on a webpage — text, images, links, buttons, and more. It is not a programming language; it just describes the structure of a page using tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

VS Code Setup

VS Code is the most popular free code editor for web development. After installing it, type ! and press Tab inside an empty .html file to auto-generate the basic HTML boilerplate shown above. Installing the 'Live Server' extension lets you preview your page in the browser and see changes instantly every time you save.

- Live Server — auto-refreshes the browser when you save a file.
- Prettier — automatically formats/cleans up your code.
- Auto Rename Tag — renames the closing tag automatically when you edit the opening tag.

Basic Page Structure

Every HTML page follows the same skeleton. The <head> section holds information about the page that is not shown directly on screen, while the <body> section holds everything the visitor actually sees.

Tag	Purpose
<!DOCTYPE html>	Tells the browser this is an HTML5 document.

<code><html></code>	The root element that wraps the whole page.
<code><head></code>	Metadata: title, character set, linked CSS files, etc.
<code><meta charset='UTF-8'></code>	Sets the character encoding so text/symbols display correctly.
<code><title></code>	Text shown on the browser tab.
<code><body></code>	All visible content of the page.

♦ HTML Text — Paragraphs & Text Formatting

Headings & Paragraphs

Headings give structure and hierarchy to a page, while paragraphs hold blocks of normal text.

Tag	Use
<code><h1></code> to <code><h6></code>	Headings — h1 is the biggest/most important, h6 the smallest. Use only one <code><h1></code> per page.
<code><p></code>	A paragraph of text.

Text Formatting Tags

These tags change how a piece of text looks or is understood by the browser/screen readers. `<strong>` and `<em>` are 'semantic' — they tell the browser the text is genuinely important or emphasized, which also helps accessibility and SEO, while `<b>` and `<i>` only change the look with no extra meaning.

Tag	Description
<code></code>	Bold text (visual only, no importance)
<code></code>	Bold text AND marks it as important
<code><i></code>	Italic text (visual only)
<code></code>	Italic text AND marks it as emphasized
<code><small></code>	Smaller font size
<code><mark></code>	Highlights text
<code>
</code>	Line break

```
<p>This is <strong>important</strong> and <em>emphasized</em>.</p>
```

Block vs Inline Elements

Every HTML element behaves as either 'block' or 'inline'. Understanding this helps you predict how elements will lay out on the page.

Type	Behaviour	Examples
Block	Starts on a new line and takes up the full width available	<div>, <p>, <h1>
Inline	Flows within the surrounding text, only as wide as its content	, , <i>, <a>

♦ HTML Essentials — Lists, Links, Images & Buttons

Lists

Lists group related items together. Use an ordered list when the sequence matters (steps, rankings) and an unordered list when it doesn't (features, ingredients).

Tag	Description
	Unordered list (bullet points)
	Ordered list (numbered)
	A single list item, used inside or

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Links (Anchor Tag)

The <a> tag creates a hyperlink that takes the user to another page, another website, or another section of the same page.

Attribute	Description
href	The destination URL

<code>target='_blank'</code>	Opens the link in a new browser tab
------------------------------	-------------------------------------

```
<a href="https://example.com" target="_blank">Visit Example</a>
```

Images

The `<img>` tag embeds a picture on the page. It is a self-closing tag — it has no separate closing tag.

Attribute	Description
<code>src</code>	Path or URL of the image file
<code>alt</code>	Text shown if the image fails to load, and read aloud by screen readers

```

```

Buttons

A `<button>` is used to trigger an action (like submitting a form or running JavaScript), while a styled `<a>` tag is used for actual navigation to another page. Choosing the right one matters for accessibility.

```
<button>Click Me</button>
```

◆ HTML Forms

Form Basics & Input Types

A `<form>` collects information from the user (like a login or contact form) and sends it somewhere using the action and method attributes. Inside a form, different input types are used depending on what kind of data you want to collect.

Input Type	Used For
<code>text</code>	Single line of text
<code>password</code>	Hides typed characters
<code>email</code>	Validates an email format
<code>number</code>	Only accepts numbers
<code>date</code>	Date picker
<code>checkbox</code>	Multiple selectable options
<code>radio</code>	One choice out of a group

file	File upload
submit	Submits the form

```
<form>
  <input type="text" placeholder="Username">
  <input type="password" placeholder="Password">
  <button type="submit">Login</button>
</form>
```

Select, Option & Labels

A `<select>` creates a dropdown list, and each choice inside it is an `<option>`. A `<label>` connects descriptive text to an input using the `for` attribute, which improves accessibility (clicking the label focuses the input).

```
<label for="country">Country:</label>
<select id="country">
  <option value="bd">Bangladesh</option>
  <option value="in">India</option>
</select>
```

♦ Learn and Explore CSS

What is CSS & Ways to Add It

CSS (Cascading Style Sheets) controls how HTML elements look — colours, spacing, fonts, layout, and more. There are three ways to add CSS to a page.

Type	How to Use
Inline	style attribute directly on a tag, e.g. <code><h1 style='color:red;'></code>
Internal	A <code><style></code> block placed inside <code><head></code> , affects only that page
External	A separate .css file linked with <code><link rel='stylesheet' href='style.css'></code> , reusable across many pages

```
selector {
  property: value;
}
```

Text Styling & Units

CSS properties can style text directly, and values are measured using different units depending on whether you want a fixed size or a size that scales with something else.

Unit	Meaning
px	Fixed pixels — doesn't scale
%	Percentage of the parent element
em	Relative to the parent's font-size (can compound when nested)
rem	Relative to the root (<html>) font-size — predictable and commonly preferred

```
h1 {  
  color: blue;  
  font-size: 2rem;  
  text-align: center;  
  font-weight: bold;  
}
```

CSS Selectors

A selector tells the browser which HTML elements a style rule should apply to. Selectors can target elements by tag name, class, id, or a combination.

Selector	Example	Selects
Type	<code>h1 {}</code>	Every <h1> element
Class	<code>.box {}</code>	Every element with class='box'
ID	<code>#header {}</code>	The one element with id='header' (must be unique)
Universal	<code>* {}</code>	Every element on the page
Grouping	<code>h1, p {}</code>	Both <h1> and <p> elements
Attribute	<code>input[type='text']</code>	Inputs whose type attribute is text

Combined / Nested Selectors

These selectors let you target elements based on their relationship to other elements, giving you much more precise control.

Selector	Description
<code>.btn.primary</code>	Element that has BOTH the btn and primary class
<code>div span</code>	Any <code></code> that is anywhere inside a <code><div></code> (descendant)
<code>p > span</code>	A <code></code> that is a DIRECT child of a <code><p></code>
<code>a[href^='https']</code>	Links whose href starts with 'https'

♦ CSS Box Model & Spacing

Margin, Padding & Border

Every HTML element is treated as a rectangular box. Padding is space INSIDE the border (between the border and the content), margin is space OUTSIDE the border (between the element and its neighbours), and border is the line drawn around the padding.

Property	Description
<code>margin</code>	Space outside the element, pushes other elements away
<code>padding</code>	Space inside the element, between content and border
<code>border</code>	A visible line around the padding
<code>border-radius</code>	Rounds the corners of the border

```
.box {  
  padding: 20px;  
  border: 1px solid #000;  
  margin: 10px;  
  width: 200px;  
}
```

The Box Model & box-sizing

By default, width and height only apply to the content — padding and border add extra size on top of that, which can make layouts tricky. Setting `box-sizing: border-box;` makes width/height include the padding and border, which is far more predictable and is used in almost every modern project.

```
* {  
  box-sizing: border-box;  
}
```

♦ CSS Display, Visibility & Backgrounds

Display & Visibility

The display property controls how an element behaves in the page layout. `visibility: hidden` hides an element but keeps the empty space it would have taken, while `display: none` removes it completely as if it never existed.

Value	Description
<code>block</code>	Full width, starts on a new line
<code>inline</code>	Flows with text, no width/height control
<code>inline-block</code>	Flows like inline but you CAN set width/height
<code>none</code>	Completely hides the element and removes its space
<code>flex / grid</code>	Turns the element into a flexible layout container

```
box-shadow: 0 4px 6px rgba(0, 0, 0, 0.2);
```

Backgrounds

Background properties let you add solid colours or images behind an element's content.

Property	Example
background-color	<code>background-color: yellow;</code>
background-image	<code>background-image: url('bg.jpg');</code>
background-size	<code>cover, contain, or 100% 100%</code>
background-position	<code>center, top right, 0 0</code>

background-repeat	no-repeat, repeat-x, repeat-y
-------------------	-------------------------------

```
body {
  background-image: url("img/bg.jpg");
  background-size: cover;
  background-position: center;
}
```

◆ Version Control with Git & GitHub

What is Version Control? Git vs GitHub

A Version Control System (VCS) keeps track of every change made to your code over time, so you can review the history or undo mistakes. Git is the tool that runs on your own computer to do this. GitHub is a website that stores your Git repositories online and adds collaboration tools on top (issues, pull requests, etc). In short: Git is the tool, GitHub is where you host your work.

Term	Description
Git	Command-line tool that tracks and manages code versions locally
GitHub	Cloud platform that stores Git repositories and enables collaboration

Creating a GitHub Repository

A repository ('repo') is simply a project folder that GitHub tracks. To create one: go to github.com, click 'New repository', give it a name, choose Public or Private, and click Create. You can then copy its link (HTTPS or SSH) to connect it to your local project.

Basic Git Commands

These four commands form the everyday backbone of using Git on your own computer.

Command	Description
git init	Initializes a new local Git repository in the current folder
git status	Shows which files have changed and whether they are staged

<code>git add .</code>	Stages all changed files, preparing them for a commit
<code>git commit -m 'message'</code>	Saves a snapshot of the staged changes with a description

```
git init
git status
git add .
git commit -m "Initial commit"
```

Setting Up Your GitHub Account for the Terminal

Before committing for the first time, Git needs to know who you are, so your commits are correctly attributed to you.

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git --version
```

VS Code Terminal & GitHub Desktop (Optional)

You don't have to leave your editor to use Git — VS Code has a built-in terminal (open with Ctrl/Cmd + `) where you can run all the same Git commands. If you prefer a visual interface instead of typing commands, GitHub Desktop offers buttons and diffs for staging, committing, and pushing.

Sending Changes to GitHub (git push)

Once you've made commits locally, you push them to GitHub so they're backed up online and visible to others.

Command	Description
<code>git remote add origin <url></code>	Connects your local repo to the GitHub repository
<code>git push -u origin main</code>	Uploads your commits the first time and remembers the connection
<code>git push</code>	Uploads new commits on every push after the first
<code>git clone <url></code>	Downloads a copy of an existing GitHub repo to your computer

Common Beginner Issues

A few small mistakes cause most of the confusion beginners run into with Git.

Issue	Solution
Wrong branch name	Rename it to main, or push it as a new branch
Permission denied	Set up HTTPS credentials or SSH keys correctly
Changes not committed	Run <code>git add .</code> before <code>git commit</code>
Push rejected	Run <code>git pull</code> first to sync remote changes, then push again

```
git remote -v
```

The Basic Git Workflow (Summary)

This five-step cycle is what you will repeat almost every time you work on a project.

```
git init
git add .
git commit -m "Your message"
git remote add origin <repo link>
git push -u origin main
```

The two topics below are advanced and optional for beginners — they can be revisited once you are comfortable with the basic Git workflow above.

[Advanced] Working with Multiple Developers on a Private Repo

In a private repository, the owner adds collaborators from Settings → Collaborators. Each collaborator then clones the repo and should work on their own branch, so their changes don't directly overwrite someone else's work.

[Advanced] Branches, Pull Requests & Merging

A branch is an independent line of work — it lets you build a feature or fix a bug without touching the main code until you're ready. A Pull Request (PR) proposes merging your branch into another branch on GitHub, so teammates can review and comment before it's combined. `git merge` is the command that actually combines the histories of two branches.

Command	Description
<code>git branch <name></code>	Creates a new branch

git checkout -b <name>	Creates AND switches to a new branch in one step
git merge <branch>	Combines another branch's changes into the current branch

♦ HTML5, Media & Semantic Tags

HTML5 vs HTML

HTML5 is the modern version of HTML. It introduced semantic tags (like <header> and <section>), native support for audio and video without needing plugins like Flash, and generally cleaner syntax.

Embedding Audio, Video & YouTube

HTML5 lets you play media natively and embed external content like YouTube videos using an iframe.

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
</audio>

<video width="320" controls>
  <source src="video.mp4" type="video/mp4">
</video>

<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  allowfullscreen></iframe>
```

Semantic HTML5 Layout Tags

Semantic tags describe the MEANING of a section, not just its appearance. Using them instead of generic <div>s everywhere makes your page easier to read, better for SEO, and more accessible to screen readers.

Tag	Purpose
<header>	Introductory content — usually a logo and/or navigation
<nav>	A block of navigation links

<code><main></code>	The main unique content of the page (only one per page)
<code><section></code>	A themed grouping of content, usually with its own heading
<code><article></code>	Independent, self-contained content like a blog post
<code><footer></code>	Closing information — copyright, contact links, etc.

```

<article>
  <header><h2>Blog Title</h2></header>
  <p>Post content ... </p>
  <footer><time>2025-05-27</time></footer>
</article>

```

♦ Advanced HTML Forms & Tables

Fieldset, Legend, Radio, Checkbox & Textarea

These elements help build richer, well-organised forms. `<fieldset>` groups related fields together visually and semantically, with `<legend>` giving that group a title.

Element	Use
<code><fieldset></code>	Groups related form controls together
<code><legend></code>	Caption/title for a fieldset
<code><input type='radio'></code>	Pick ONE option — radios sharing the same name are grouped
<code><input type='checkbox'></code>	Pick multiple independent options
<code><textarea></code>	Multi-line free text input
<code><input type='reset'></code>	Clears all fields back to default
<code><input type='submit'></code>	Submits the form

```

<form>
  <fieldset>
    <legend>Survey</legend>

```

```

    <label>Name: <input type="text" name="name"></label><br>
    <label><input type="radio" name="gender" value="M"> Male</label>
    <label><input type="radio" name="gender" value="F"> Female</label><br>
    <textarea rows="4" cols="30"></textarea>
    <input type="submit">
  </fieldset>
</form>

```

HTML Tables

Tables display data in rows and columns — useful for things like price lists, schedules, or comparison charts (not for page layout, which is what CSS is for).

Tag	Description
<code><table></code>	Wraps the entire table
<code><tr></code>	A table row
<code><th></code>	A header cell (bold, centered by default)
<code><td></code>	A normal data cell
<code><caption></code>	A title for the whole table
<code><thead></code> / <code><tbody></code> / <code><tfoot></code>	Groups the header, body, and footer rows
<code>colspan</code> / <code>rowspan</code>	Merges a cell across multiple columns/rows

```

<table>
  <caption>Student Marks</caption>
  <thead><tr><th>Name</th><th>Score</th></tr></thead>
  <tbody><tr><td>Ali</td><td>95</td></tr></tbody>
</table>

```

◆ CSS Flexbox & Page Sections

Flexbox Basics

Flexbox is a layout system that makes it easy to arrange items in a row or column, and to align/distribute the space between them — perfect for navbars, cards, and centering content.

Property	Description
----------	-------------

<code>display: flex</code>	Turns a container into a flex container
<code>flex-direction</code>	row (default) or column — sets the main axis
<code>justify-content</code>	Aligns items along the main axis (center, space-between, etc.)
<code>align-items</code>	Aligns items along the cross axis (center, stretch, etc.)
<code>flex-wrap: wrap</code>	Lets items move to a new line when they run out of space
<code>gap</code>	Adds spacing between flex items without using margins

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}
```

Centering with margin: auto

Setting a fixed width and margin: 0 auto on a block-level element automatically centers it horizontally on the page — a simple trick used constantly in real layouts.

```
.container {
  width: 500px;
  margin: 0 auto;
}
```

Navbars & Internal Navigation

A navbar is usually a `<nav>` containing a list of links, laid out horizontally with flexbox. You can also link to a specific section on the SAME page using an id and a # in the href.

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="#contact">Contact</a>
</nav>
```

```
<section id="contact"> ... </section>
```

Hero Sections

A 'hero' is the large introductory banner at the top of a page, usually with a heading, short text, and a button. Flexbox is commonly used to center or split the hero's content.

```
<section class="hero">
  <div class="hero-text">
    <h1>Welcome</h1>
    <p>Your site starts here.</p>
  </div>
  
</section>
```

```
<style>
.hero {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 50px;
}
</style>
```

Card Components

A 'card' is a small, self-contained box (image + title + text + button) commonly used for products, blog previews, or team members. Multiple cards are usually laid out side-by-side with flexbox and flex-wrap so they reflow on smaller screens.

◆ CSS Units, Grid & Positioning

CSS Units Recap

Choosing the right unit affects how well your design adapts to different screens and user settings.

Unit	Description
px	Absolute, fixed size

%	Relative to the parent element
em	Relative to the parent's font-size, compounds when nested
rem	Relative to the root font-size — consistent and predictable
auto	Browser calculates the value automatically

CSS Grid Basics

While flexbox is great for a single row or column, Grid is built for two-dimensional layouts — rows AND columns at the same time, which makes it ideal for things like photo galleries or calendars.

Property	Description
<code>display: grid</code>	Turns a container into a grid container
<code>grid-template-columns</code>	Defines the columns, e.g. <code>repeat(3, 1fr)</code> for 3 equal columns
<code>gap</code>	Spacing between grid cells
<code>grid-column</code> / <code>grid-row</code>	Makes a cell span multiple columns/rows

```
.calendar {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  gap: 8px;
}
```

CSS Positioning

The position property controls exactly where an element sits, either in the normal page flow or pulled out of it.

Value	Behaviour
static	Default — normal flow, top/left/etc. have no effect
relative	Shifted relative to its OWN normal position, still keeps its original space

absolute	Removed from flow, positioned relative to the nearest positioned ancestor
fixed	Positioned relative to the browser window — stays in place when scrolling
sticky	Acts like relative until a scroll point, then behaves like fixed

```
.sticky-header {  
  position: sticky;  
  top: 0;  
}
```

z-index (Stacking Order)

When elements overlap, z-index decides which one appears on top — a higher number sits closer to the viewer. It only has an effect on elements that are positioned (not static).

```
.modal {  
  position: absolute;  
  z-index: 10; /* appears above elements with a lower z-index */  
}
```

◆ Advanced CSS Topics (with Video Resources)

These five topics extend the layout and styling skills above. Each includes a simple explanation, an example, and beginner-friendly video tutorials in English, Hindi, and Bangla.

1. Pseudo Class

A pseudo-class (written with a single colon :) selects an element based on its current STATE or POSITION, without needing to add an extra class in your HTML. For example, you can style a button differently the moment the mouse hovers over it, or style every other row in a list.

- :hover — when the mouse is over the element
- :focus — when an input/element is actively selected
- :visited — a link the user has already clicked before
- :first-child / :last-child / :nth-child(n) — target elements by position among siblings
- :not() — selects everything EXCEPT what matches inside the parentheses

```
/* Button changes color when hovered */

button:hover {

    background-color: darkblue;

    color: white;

}

/* Input gets a green border when focused */

input:focus {

    border: 2px solid green;

    outline: none;

}

/* Every odd row gets a light gray background (like a striped table) */

li:nth-child(odd) {

    background-color: #f5f5f5;

}

/* Style all list items except the first one */

li:not(:first-child) {
```

```
margin-top: 10px;

}
```

Language	Video Resource
English	HTML & CSS Crash Course – Pseudo Classes & Elements (Web Dev Simplified)
Hindi	CSS Pseudo-classes – Web Development in Hindi (CodeWithHarry)
Bangla	CSS Pseudo Classes In Bangla (Web Ground)

2. Pseudo Element

A pseudo-element (written with a double colon ::) targets a specific PART of an element, rather than the whole thing — like just the first letter of a paragraph, or content inserted before/after an element.

::before and ::after are extremely common for adding decorative icons or shapes without writing extra HTML.

- ::before / ::after — insert generated content before or after an element's content (requires a content property)
- ::first-letter — styles just the first letter of a block of text
- ::first-line — styles just the first line of a block of text
- ::selection — styles text the user has highlighted with their mouse

```
/* Add a star icon before every card title, without touching the HTML */

.card-title::before {

    content: "★ ";

    color: gold;

}

/* Drop-cap effect: make the first letter big */

p::first-letter {

    font-size: 250%;

    font-weight: bold;

    color: crimson;

}
```

```
/* Style whatever text the user selects/highlights */
```

```
::selection {  
  
    background: yellow;  
  
    color: black;  
  
}
```

```
/* Style input placeholder text */
```

```
input::placeholder {  
  
    color: #999;  
  
    font-style: italic;  
  
}
```

Language	Video Resource
English	Learn CSS Pseudo Elements In 8 Minutes (Web Dev Simplified)
Hindi	Before and After Pseudo Selectors – Web Development in Hindi (CodeWithHarry)
Bangla	CSS Pseudo Classes And Elements Bangla Tutorial (Web Ground)

3. Translate, Transform, Scale, Rotate

The transform property visually moves, resizes, or rotates an element WITHOUT affecting the layout of the elements around it. It's one property that accepts several different functions, which can even be combined together in a single line.

- `translate(x, y)` — moves the element from its current position
- `scale(x, y)` — resizes the element bigger or smaller
- `rotate(deg)` — spins the element around a fixed point (default: its center)
- `skew(x, y)` — slants the element along the X and/or Y axis
- Modern CSS also allows translate, scale, and rotate to be used as their OWN separate properties, not just inside transform.

```
/* Move a box 50px right and 20px down */
```

```
.box1 {  
  
    transform: translate(50px, 20px);  
  
}
```

```
}

/* Rotate an image 15 degrees clockwise */

.tilted-photo {
    transform: rotate(15deg);
}

/* Grow a card to 1.1x its size on hover – nice interactive effect */

.card {
    transition: transform 0.3s ease;
}

.card:hover {
    transform: scale(1.1);
}

/* Combine multiple transforms in one line */

.combo {
    transform: translateX(20px) rotate(10deg) scale(1.2);
}


.box2 {
    translate: 20px 0;
    rotate: 10deg;
    scale: 1.2;
}
```

Language	Video Resource
English	Master CSS Transform Property: Rotate, Scale, Translate & Skew Tutorial
Hindi	CSS Transform 2D Tutorial in Hindi – Translate, Rotate, Skew, Scale (Rahul Chaudhary)
Bangla	CSS3 Full Course Bangla Tutorial (covers Transform) – Anisul Islam

4. Animation

CSS animations let an element gradually change its style over time, entirely without JavaScript. You first define the stages of the animation using `@keyframes`, then attach that animation to an element using the animation properties.

- `@keyframes` — defines what the element looks like at each stage (0% = start, 100% = end)
- `animation-name` / `animation-duration` — which keyframes to use, and how long one cycle takes
- `animation-iteration-count` — how many times it repeats (or infinite)
- `animation-timing-function` — the speed curve, e.g. ease, linear
- `animation` — a shorthand that combines all of the above in one line

```
/* Define the animation stages */

@keyframes bounce {

  0%   { transform: translateY(0); }

  50%  { transform: translateY(-20px); }

  100% { transform: translateY(0); }

}

/* Apply it to an element */

.ball {

  width: 40px;

  height: 40px;

  background: tomato;

  border-radius: 50%;
```

```

    animation: bounce 1s ease-in-out infinite;
}

/* A fade-in animation, common for page/section loading */
@keyframes fadeIn {
    from { opacity: 0; }
    to   { opacity: 1; }
}

.hero {
    animation: fadeIn 1.5s ease forwards;
}

```

Language	Video Resource
English	CSS Animations Tutorial – Keyframes + Bouncing Ball Example
Hindi	Creating Animations & Keyframes – Web Development in Hindi (CodeWithHarry)
Bangla	CSS Animation Using Keyframes – Bangla Tutorial

5. Media Query

A media query applies CSS rules only when certain conditions are met — most commonly the width of the browser window. This is the foundation of responsive design, letting one website adapt its layout for phones, tablets, and desktops.

- Mobile-first approach — write base styles for small screens first, then add complexity for larger screens with min-width queries.
- Common breakpoints target mobile, tablet, and desktop widths, though modern practice lets the CONTENT decide where breakpoints go.
- Related modern tools: clamp(), min(), max() and container queries can sometimes reduce how many media queries you need.


```
/* Default styles – written for mobile first */

.navbar {

    display: flex;

    flex-direction: column;
}

.card-container {

    display: grid;

    grid-template-columns: 1fr; /* 1 column on small screens */
}

/* When the screen is 768px wide or larger (tablet and up) */
@media (min-width: 768px) {

    .navbar {

        flex-direction: row;
    }

    .card-container {

        grid-template-columns: repeat(2, 1fr); /* 2 columns */
    }
}

/* When the screen is 1024px wide or larger (desktop) */
@media (min-width: 1024px) {

    .card-container {

        grid-template-columns: repeat(3, 1fr); /* 3 columns */
    }
}
```

Language	Video Resource
English	CSS Media Queries & Responsive Web Design Tutorial for Beginners (Dave Gray)
Hindi	CSS Media Query in Hindi – Responsive Web Design
Bangla	CSS Media Queries Bangla Tutorial – Responsive Web Design (Web Ground)